

Maxim Dokukin
017025164
CMPE 259
Prof Mahima Agumbe Suresh
05/16/2026

Project Report

1. Motivation

My website has a lot of data (60 projects, 10 work experiences, 3 educations), Some users might never find information they are looking for. This problem matters because it makes the information on my website more accessible to the users, therefore improving my chances of spreading the word about my projects/work experience.

I believe that shifting from the “search-through information” user experience to “evaluate-the-presented information” ux is a trend that we saw over the past years, which is only accelerating. I like to bring up an example of someone looking into the code documentation VS using a LLM chat bot to look it up. I don't know too many people who are still looking through the docs themselves.

As the outcome, this project can also become a foundation of a personal assistant (intelligent personal information hub) that acts semi-autonomously on my behalf. For instance, drafting email responses, and confirming arrangements based on my data and preferences.

2. VA requirements:

I am imagining three types of users: professional (HRs), hobbyists, friends. For the professional people the secretary should be able to accurately summarize the work/project experience for the inquired area, and provide references to the relevant work. For the hobbyists, it should make it easier to navigate my projects, and allow to go straight to the project page. For friends, it is just a fun thing to see and try to hack it with inappropriate queries.

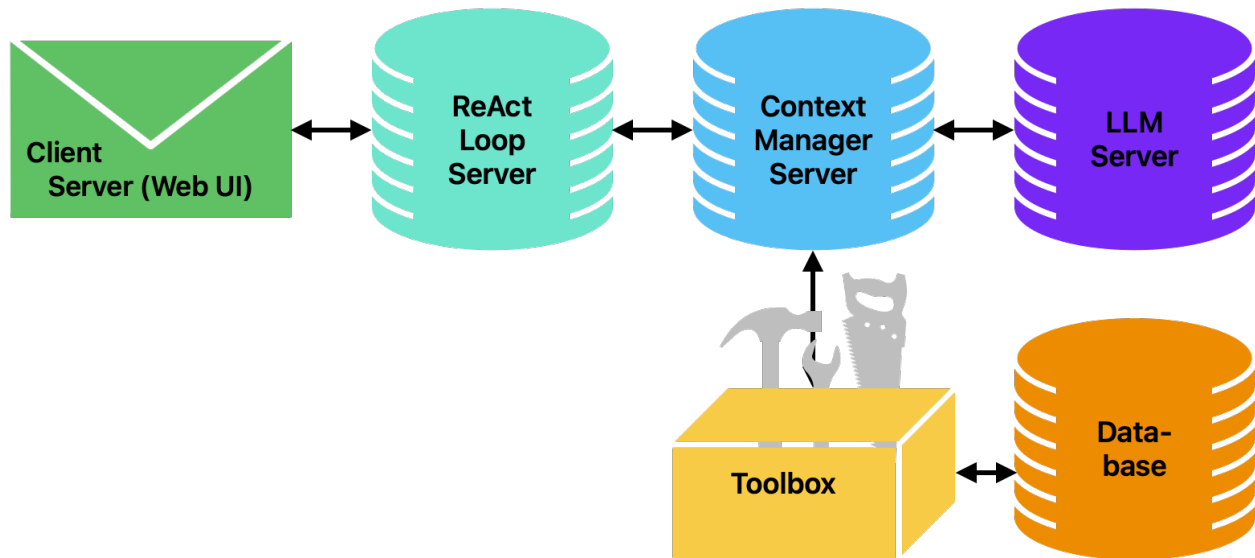
3. Security considerations

Since all the information that the secretary has access to is publicly available, there is no major security considerations. However, I still saw VA giving away table schemas and raw html values which might not be ideal. I have considered introducing another LLM-data-fetcher that would isolate raw data from the final secretary-LLM that users interface with. However that would add additional latency, compute, and latency to the responses.

Another threat front that is worth mentioning is prompt injections. While my VA has no access to executing any queries that modify or delete the data, I still should be conscious around that to prevent potential SQL injections that would corrupt the data.

To conclude, due to a limited exposure to private data, and no ability to execute tools that would harm the data, security considerations here are at low stakes.

4. Design of the VA



The design consists of the following elements:

- ClientServer (for chat interface)
- ReAct Loop (for handling reasoning and tool calls)
- ContextManager (for managing data and message history)
- ToolBox (for tools that retrieve data)
- LLMServer (Gemma 4 E4B and GEMma 4 26B)

There are two main reasons why I went with multiple server architecture: structured and organized access to each part of the data flow, and system modularity (each component can be local or remote).

Data flow observability was necessary for me to trace how each component interacts. That allowed me to make sure that the LLM Server receives only necessary data, while the ContextManager Server keeps track of the sessions and saves them as JSONS.

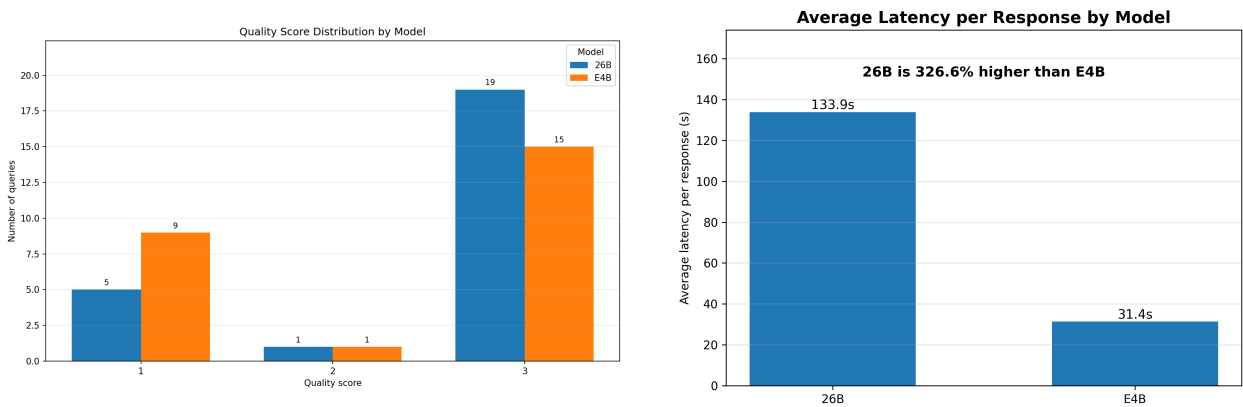
On the modularity, I recognized that some components will be local and some will be in the cloud. For example, the DB is a supabase, and I don't have need to have it locally. If fact, I need it to be in the cloud so that my website cloud host can have access to the data at all times. But the LLM Server can either run locally or remotely. I utilized another machine on my WiFi for LLM Server during the testing, and was able to easily connect to it instead of pushing my laptop's thermal capabilities. This server-architecture modularity in general allows for a more flexible system, while keeping the structure in place.

On the component level: ClientServer is the interface between the user and the system. ReAct Loop Server is necessary for the agent to be able to handle a sequence of tool calls, reasoning iterations, and final answer generation. ContextManager is a part that keeps track of what kind of data is present in the context. ToolBox defines available tools that VA can use to improve the results. And LLMServer is there to handle the actual inference.

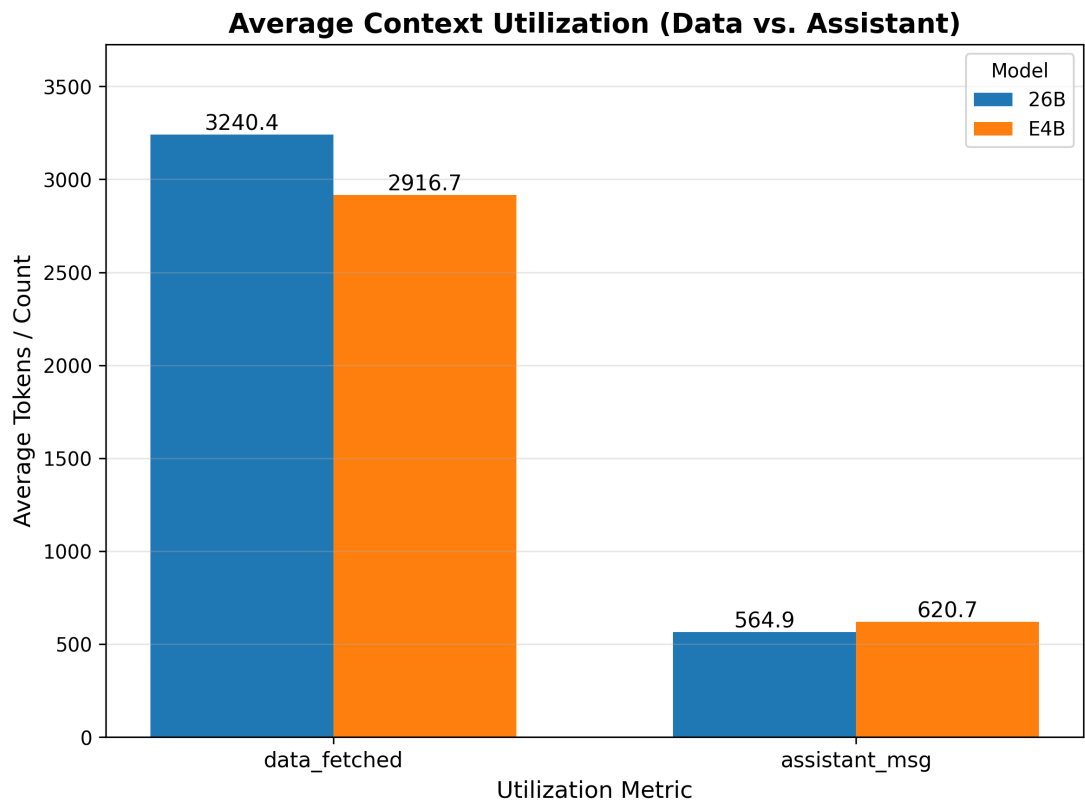
Alternatively, this project could have been implemented as a single server application. But in this case, it would have been much harder to host these components as a mix of local and cloud, and to trace the data flows. Moreover, the single server approach would also noticeably impair codebase organization and code reusability.

5. Results

The results have been surprising to me. The 26B and 4B models were very close in performance. Both of them had the hardest time in writing SQL queries for the query execution tool. I believe that the 26B performance win comes mostly from the ability of writing better queries. But this accuracy came at a high latency increase (114 seconds for 26B and 32 seconds for E4B) on average per response. Summarizing, both models perform well within the framework, with bigger model doing a little better on tool calls due to a better SQL query generation.



I have noticed that the model utilized the context differently, with 26B fetching more data per response, while E4B provided longer final responses. This is also likely due to a bigger model performing better on query writing, and therefore being able to fetch more data successfully.



6. Reflection

I am impressed with the capabilities of the open source models that can run on my laptop. I have plans for taking my present secretary (currently using OpenAI API <https://maxdokukin.com/>) locally for the sake of data privacy.

I think that my current VA prototype is capable, but needs more work on better context management and tool calling logic. Context management is the area that I want to extend this project in, specifically on removing old/irrelevant data from the context. I am wondering if I could somehow utilize the attention mechanism to help with determining what parts of the context are more relevant and are worth keeping, while recognizing unattended parts and discarding them.

Tool calling logic currently struggles from LLMs writing queries that don't produce the expected results. I think there are two ways to go here: first is to improve the existing SQL logic; second is to experiment with the vector store. At this point I am not sure what is a better approach. SQL seems to be an easier integration in my existing data model, and I am familiar with the technology. While vector store can potentially provide better retrieval, information embeddings is still a dark area for me, and I am not sure if that would be the right tool for my use case.

The main learning points for me were: running local LLMs, and inter-server communications with REST API. I now can confidently build llama.cpp and design communication paradigms between the servers.

If I were to take it further, I would focus on these improvements:

1. Data retrieval improvements either by better SQL retrievals, or vector store
2. Experiments with how attention mechanism can be ties together with the context management
3. Hosting the LLM server in the cloud that has zero data retention policies (AWS/Oracle)

7. References (does not count in the page limit): List ALL sources and resources you used to complete the project, including AI tools, grammar checkers, and IDEs.

ClientServer, ContextManager, LLMServer

Some functions implemented with assistance of Gemini, styling is completely AI implementation.

Back end FAST Api, llama.cpp

Data is hosted on Supabase

IDE/Tools: JetBrains DataGrip, PyCharm, ssh, vi, zsh

With these tools I mainly focus on the idea, system architecture, data model development, and integration, while leaving the code implementation to the Gemini.